폼 액션의 서버 함수와 클라이언트 상태

- 날짜: 2026-11-02
- 대상: Next.js 16.4.3, React 19.3.0, `lgcns-fe-next`
- 기준 파일은 현재 소스다. `/login`은 서버 컴포넌트 안의 서버 함수가 `FormData`를 읽는다. `/client-login`은 클라이언트 컴포넌트가 `useState` 값을 브라우저에서만 읽는다.

이 두 화면은 로그인 입력을 다루지만, 값이 움직이는 곳이 다르다. `/login`의 제출은 서버 함수다. `/client-login`의 제출은 브라우저 이벤트이고, `fetch`나 `XMLHttpRequest`는 그 파일에 없다.

1. 파일과 선언

파일	선언	모듈	하는 일
app/(auth)/login/page.tsx	async function handleSubmit + "use server"	서버 컴포넌트	FormData에서 userId, password를 읽어 서버 로그에 찍는다
component/templates/LoginSection.tsx	function LoginSection	서버 컴포넌트	서버 함수를 유기체로 넘기는 <section>
component/organisms/LoginForm.tsx	"use client" + function LoginForm	클라이언트 컴포넌트	useState로 값과 오류를 두고, next/form의 action에 서버 함수를 넣는다
component/molecules/FormField.tsx	function FormField	클라이언트 그래프	지시문은 없다. LoginForm이 가져오므로 클라이언트 번들에 포함된다. name이 있는 입력을 그린다
component/atoms/ErrorMessage.tsx	function ErrorMessage	클라이언트 그래프	지시문은 없다. FormField가 가져온다. 오류 문자열이 있을 때만 문단을 그린다
schema/loginSchema.ts	function loginFieldMessage	클라이언트 그래프	LoginForm만 가져온다. 브라우저에서 Zod로 필드 메시지를 만든다. 서버 함수는 이 파일을 호출하지 않는다
types/request/RequestType.ts	interface UserPayloadType	타입	userId, password가 string이다. 런타임 값은 아니다
app/(auth)/client-login/page.tsx	'use client' + function ClientLogin	클라이언트 컴포넌트	입력마다 useState를 바꾸고, 제출 시 기본 동작을 막은 뒤 그 상태를 브라우저 로그에 찍는다
app/(auth)/layout.tsx	function AuthLayout	서버 컴포넌트	두 페이지를 감싼다
components/ui/input.tsx	function Input	클라이언트 경계	파일에 지시문은 없다. @base-ui/react/input의 Input.js 첫 줄이 'use client'다
components/ui/button.tsx	function Button	클라이언	@base-ui/react/button의 Button.js 첫 줄이 'use client'다

파일	선언	모듈	하는 일
		트 경계	
next/form	import Form from "next/form"	클라 이언 트 모듈	next/dist/client/form.js 첫 줄이 'use client'다

(auth)는 라우트 그룹이다. URL에는 없다. 주소는 /login과 /client-login이다.

handleSubmit의 async는 ECMAScript async 함수다. 호출 결과가 프로미스다. HTML의 <script async>와 같은 속성이 아니다. Next.js 문서에서 서버 함수는 비동기 함수여야 한다. 서버에서 실행되고, 클라이언트가 네트워크 요청으로 부르기 때문이다.

"use server"는 그 함수를 서버 함수로 표시하는 지시문이다. 이 페이지에서는 함수 본문 첫 줄에 있다. "use client"는 그 파일이 클라이언트 컴포넌트 그래프의 시작이라는 지시문이다. 두 지시문은 호이스팅 문법이 아니다.

2. 로딩 시점

이 로그인 파일은 HTML에 <script src>로 적혀 있지 않다. Next가 서버에서 서버 컴포넌트를 실행하고, "use client"로 시작한 그래프만 브라우저 모듈로 보낸다.

코드	적힌 곳	로딩
<script src="js/main.js">	lgcns-fe-html/index.html, about.html 본문 끝	속성 없음. 파서가 그 지점에서 스크립트를 받아 실행한 뒤 다음 HTML로 간다. 동기 실행이다
<script src="js/main.js" async>	lgcns-fe-html/contact.html	async. 받는 동안 파서는 계속 가고, 받으면 바로 실행한다. 여러 스크립트의 실행 순서는 문서 순서가 아니다
<script src defer>	이 저장소의 로그인 HTML에는 없다	defer는 받는 동안 파서를 막지 않고, 문서가 파싱된 뒤 문서 순서대로 실행한다
<script type="module" src="/src/main.tsx">	lgcns-fe/index.html	type="module". 모듈 스크립트는 지연 실행된다. defer를 적어도 모듈의 기본 동작과 같다
async function handleSubmit	app/(auth)/login/page.tsx	브라우저 스크립트 태그가 아니다. 함수 본문의 "use server" 때문에 제출 때 서버에서 실행된다
"use client" LoginForm, ClientLogin	클라이언트 컴포넌트	Next가 클라이언트 모듈로 보낸다. onChange와 onSubmit은 브라우저에서 실행된다

서버 컴포넌트인 login/page.tsx와 LoginSection.tsx의 함수 본문은 브라우저 번들로 내려가지 않는다. 그 안에서 만든 서버 함수만, 클라이언트가 부를 수 있는 참조로 넘어간다. LoginForm이 FormField와 loginSchema를 import 하므로, 지시문이 없는 그 파일들도 클라이언트 모듈에 함께 실린다.

3. /login: 서버 함수와 FormData

페이지는 서버 컴포넌트다. 파일 맨 위에 "use client"가 없다. 서버 함수는 컴포넌트 안에 선언되고, props로 내려간다.

```
async function handleSubmit(formData: FormData) {
  "use server";
  const userId = formData.get("userId");
  const password = formData.get("password");
  console.log("userId: ", userId, "password: ", password);
}
```

LoginSection은 이 함수를 그대로 LoginForm에 넘긴다. LoginForm은 클라이언트 컴포넌트이고, next/form의 Form에 action={handleSubmit}을 넣는다. Next.js 문서는 서버 함수를 클라이언트 컴포넌트 안에 정의할 수 없고, 서버 쪽에서 만들어 props로 넘길 수 있다고 적는다. 이 화면이 그 형태다.

제출 경로는 이렇다.

1. 브라우저가 입력을 바꿀 때마다 LoginForm의 handleChange가 실행된다. 이 함수는 서버로 가지 않는다. payload 상태와 errors 상태를 갱신한다.
2. loginFieldMessage가 그 필드만 loginSchema.shape[field].safeParse 한다. 빈 문자열이면 "아이디를 입력해 주세요." 또는 "비밀번호를 입력해 주세요."를 반환한다.
3. loginSchema.safeParse(payload).success가 거짓이면 제출 버튼의 disabled가 참이다.
4. 두 값이 모두 한 글자 이상이면 버튼이 눌린다. Form의 action이 함수이므로 Next.js Form은 React 폼처럼 그 서버 함수를 실행한다.
5. 서버 함수의 인자는 React 상태 객체가 아니다. HTML 폼이 성공한 컨트롤로 만드는 FormData다. FormField의 Input에 name="userId", name="password"가 있어서 그 이름이 키가 된다.
6. formData.get은 그 키의 첫 값을 반환한다. 이 소스는 반환 타입이 string인지 검사하지 않고 console.log만 한다. 로그는 브라우저 콘솔이 아니라 서버 프로세스로 나간다. 개발 서버에서는 POST /login 다음에 userId:와 password:가 찍힌다.

입력은 통제 컴포넌트다. value={payload.userId}처럼 React 상태가 화면에 보이는 값을 정한다. 같은 입력이 name도 가지므로, 제출 시점의 DOM 값이 FormData에 들어간다. 서버 함수는 payload를 받지 않는다. schema/loginSchema.ts도 호출하지 않는다. Zod 검사는 버튼과 오류 문구를 위한 클라이언트 동작이다.

"use server"가 없는 함수를 이 폼의 action에 넣으면, 클라이언트 컴포넌트인 Input, Button, Form 쪽으로 함수가 넘어가며 프리렌더가 실패한다. 지시문이 있어야 그 함수가 서버 함수 참조가 된다.

Next.js 문서는 폼으로 호출된 서버 함수가 POST로만 호출된다고 적는다. 이 저장소의 handleSubmit은 인증, 쿠키, redirect, revalidatePath를 하지 않는다. 읽어서 로그로 남기는 것까지가 구현이다.

4. /client-login: 상태 후 브라우저에서 처리

client-login/page.tsx는 파일 첫 줄이 'use client'다. 페이지 함수 전체가 클라이언트 컴포넌트다. 서버 함수는 없다.

```
const [userId, setUserId] = useState("");
const [password, setPassword] = useState("");

const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  console.log("userId: ", userId, "password: ", password);
};
```

경로는 이렇다.

1. 키 입력마다 `onChange`가 `setUserId` 또는 `setPassword`를 호출한다. 그 전에 `e.target.value`를 브라우저 콘솔에 찍는다.
2. `value={userId}`이므로 화면 값은 React 상태가 정한다. 입력에 `name`이 없다.
3. 제출하면 `onSubmit`이 `preventDefault`를 호출한다. HTML 폼의 기본 동작인 이동과 요청 전송이 취소된다.
4. 함수는 `FormData`를 만들지 않는다. 클로저 안의 `userId`, `password` 상태를 읽는다.
5. 세 줄의 `console.log`가 브라우저 콘솔에 남는다. 서버 로그의 `POST /client-login`은 이 제출로 생기지 않는다.

React `useState`는 그 컴포넌트가 다시 렌더될 때 같은 상태 값을 돌려준다. `setUserId`를 호출하면 그 컴포넌트가 새 값으로 다시 그려진다. 이 값은 서버 컴포넌트 props로 올라가지 않는다.

이 파일을 두고 "클라이언트 통신"이라고 부르면, 통신의 끝은 브라우저 안의 이벤트와 콘솔이다. `fetch`로 JSON을 보내는 코드는 없다. MDN의 `FormData`는 `fetch`나 `XMLHttpRequest.send`에 넘길 수 있다고 적지만, `client-login/page.tsx`는 `FormData`를 쓰지 않는다.

5. 두 방식 비교

	/login	**/client-login**
페이지 선언	서버 컴포넌트. 안에 `"use server"` 함수	파일 전체 `'use client'`
폼	`next/form`의 `<Form action={handleSubmit}>`	HTML `<form onSubmit={handleSubmit}>`
값이 사는 곳	제출 값은 `FormData`. 화면 값과 오류는 `useState`	화면 값과 제출 값이 같은 `useState`
입력 이름	`name="userId"`, `name="password"`	`name` 없음
제출 시	서버 함수가 `POST`로 실행되고 서버 로그가 남는다	`preventDefault` 후 브라우저 로그만 남는다
검사	클라이언트 Zod. 실패하면 버튼 `disabled`	검사 코드 없음
서버가 받는 인자	`FormData`	없음

`/login`도 클라이언트 상태를 쓴다. 차이는 제출의 목적지가 서버 함수인지, 브라우저 이벤트에서 끝나는지다.

6. 명세와 공식 문서

HTML 폼 제출은 이름이 있는 성공한 컨트롤로 폼 데이터 셋을 만든다. `name`이 없는 컨트롤은 그 셋에 들어가지 않는다. 그래서 `/login`의 `FormField`는 `name`이 있고, `/client-login`의 입력은 상태가 값을 들고

있으므로 name이 없어도 그 페이지의 console.log는 동작한다.

FormData.get(name)은 그 키의 첫 값을 반환한다. 없으면 null이다. 텍스트 입력이면 값은 문자열이다. 명세는 XMLHttpRequest의 FormData이고, 사용 설명은 MDN FormData다. get의 반환을 문자열로 좁히는 코드는 login/page.tsx에 없다.

preventDefault는 이벤트의 기본 동작을 취소한다. 폼 submit의 기본 동작은 폼을 보내는 것이다.

React 19의 <form action>은 HTML action에 더해, 함수를 받으면 그 함수에 FormData를 넘긴다. useState는 클라이언트 컴포넌트 함수의 최상위에서 호출한다. ClientLogin과 LoginForm이 그렇게 한다.

Next.js는 이 함수를 서버 함수로 다룬다. 폼의 action이나 버튼의 formAction에 넘기면 Server Action으로 호출되고, 내부적으로 POST만 허용된다. 서버 컴포넌트 안에 인라인으로 두려면 함수 본문 첫 줄에 'use server'를 둔다. 클라이언트 컴포넌트에는 정의할 수 없고, props로 받은 서버 함수를 action에 넣을 수 있다.

import Form from "next/form"은 HTML <form>을 확장한 컴포넌트다. action이 문자열이면 GET 검색 파라미터로 이동하고, 함수이면 React 폼처럼 그 서버 함수를 실행한다. 이 프로젝트의 LoginForm은 함수를 넘긴다. 문자열 action의 prefetch는 이 화면에서 쓰이지 않는다.

스크립트 로딩 속성의 정의는 HTML script 요소다. async와 defer는 외부 스크립트의 가져오기와 실행 시점을 정한다. type="module" 스크립트는 지연된다. Next 앱 라우터의 서버 컴포넌트와 서버 함수는 그 태그를 페이지에 직접 적지 않는다.

- https://xhr.spec.whatwg.org/#interface-formdata
- https://developer.mozilla.org/en-US/docs/Web/API/FormData
- https://developer.mozilla.org/en-US/docs/Web/API/Event/preventDefault
- https://react.dev/reference/react-dom/components/form
- https://react.dev/reference/react/useState
- https://nextjs.org/docs/app/getting-started/mutating-data
- https://nextjs.org/docs/app/api-reference/components/form
- https://html.spec.whatwg.org/multipage/scripting.html#the-script-element

7. 이 소스에 없는 것

- /client-login의 fetch, XMLHttpRequest, 서버 API 호출
- handleSubmit 안의 Zod 검사, 세션 확인, 쿠키 저장, redirect, revalidatePath
- useActionState, useFormStatus로 제출 중 표시
- loginSchema를 서버 함수에서 import 하는 코드

문서에 나오는 그 패턴은 Next.js가 서버 함수로 할 수 있다고 적은 동작이다. 이 저장소의 두 로그인 파일은 그 코드를 포함하지 않는다.